TokenPropulsion

Auditable Prompt Formatting and Evidence-Bounded Optimization



Ciprian Ștefan Plești

Version 1.0.0rc1 | 17 October 2026 | MIT

Independent engineering whitepaper. Not peer reviewed.

Exactly 10,000 whitespace-delimited words in the counted body, including headings and citations; front matter and references excluded.

Local engineering evidence does not establish universal semantic equivalence or live xAI compatibility. This project is independent of xAI, SpaceX, and Elon Musk.

Contents

Abstract 3

1. Introduction and research scope 3

2. Problem formulation and operational definitions 4

3. Related work and positioning 5

4. Architecture and dependency boundaries 6

5. Protection policy and refusal behavior 7

6. Transformation profiles and mechanical validation 8

7. Token accounting and provenance 9

8. Cost and latency models 10

9. xAI adapter and explicit external communication 11

10. CLI, encoding, and audit export 12

11. HTTP service and deployment boundary 13

12. Offline dataset and baseline design 14

13. Recorded local results and their interpretation 15

14. Output evaluation and semantic claims 16

15. Experimental protocol for future remote studies 17

16. Security model and residual risk 18

17. Testing strategy and evidence quality 19

18. Reproducibility and release governance 20

19. Research agenda and conclusion 21

20. Deployment decision protocol 21

References 23

Evidence artifacts and reproducibility 23

Abstract

TokenPropulsion investigates a deliberately limited question: how can an application reduce redundant prompt formatting while preserving an inspectable account of every change and refusing transformations that exceed its evidence? The implemented answer is a dependency-free Python core combining conservative whole-prompt protection, deterministic formatting profiles, mechanical validation, reversible audit records, and explicit token-accounting provenance. Optional components provide an authenticated HTTP interface, an official xAI SDK adapter, and separate offline and remote evaluation protocols. This whitepaper presents the architecture, threat model, measurement framework, executed engineering evidence, and limitations of version 1.0.0rc1. The recorded local suite contains 191 passing tests and 99.61 percent meaningful line coverage. A nine-category synthetic benchmark reports 3,829 estimated input tokens before processing and 3,578 afterward under the configurable byte-based estimator. These values establish neither actual provider token reduction nor downstream semantic equivalence. Live xAI behavior was not measured, and remote quality, latency, and billing outcomes remain unverified. The central contribution is an auditable operational contract rather than a new learned compression algorithm. We distinguish textual invariants from task correctness, local preprocessing from remote inference, and reproducible software behavior from external validity. The resulting design favors refusal, explicit uncertainty, and independently repeatable evaluation over aggressive compression claims.

1. Introduction and research scope

Prompt optimization is often described as a straightforward reduction problem: remove input that appears unnecessary, send fewer tokens, and obtain a cheaper or faster response. That description suppresses several distinct questions. The application must determine what counts as unnecessary, which tokenizer defines the reduction, whether a changed instruction still expresses the intended task, and whether the complete system actually improves after preprocessing overhead is included. A smaller string can be a worse instruction. A smaller estimated count can correspond to an unchanged provider bill. A successful example can conceal systematic failure on numerical, multilingual, or exact-output tasks. Engineering practice therefore requires a contract stronger than an attractive compression ratio.

TokenPropulsion starts from the observation that a narrow transformation can still be useful when its limits are explicit. Some application-generated prompts contain redundant blank lines, terminal spaces, or repeated ordinary spaces in prose. Removing those characters may simplify input without deleting words. However, the same characters can encode Markdown hard breaks, indentation, literal string contents, layout requirements, or source-code semantics. The project consequently treats formatting optimization as a conditional operation, not as a universally safe property of whitespace. Detection, validation, accounting, and disclosure are part of the operation itself.

The implemented release makes three practical commitments. First, the core never invokes a language model or external compression service. Second, recognized risky syntax disables transformation for the entire prompt. Third, every successful transformation remains reversible because the result retains the original input. These commitments reduce some classes of implementation risk, but they do not establish that an arbitrary unrecognized prompt is semantically invariant under normalization. The distinction is fundamental to the interpretation of every result reported here.

This document is an engineering whitepaper rather than a peer-reviewed experimental paper. Its author is Ciprian Ștefan Plești, and the original project is distributed under the MIT license. No university, laboratory, vendor, or standards organization is represented as endorsing the work. The project is independent of xAI, SpaceX, and Elon Musk. References to vendor interfaces identify optional interoperability targets; they do

not imply affiliation or privileged access. The analysis concerns the inspected release candidate, not unspecified future versions.

The research questions are correspondingly modest. Can the software enforce ordered textual invariants across supported formatting profiles? Can it preserve recognized structured and security-sensitive inputs byte for byte? Can users distinguish estimated accounting from provider-returned usage? Can the package be built, installed, tested, and reviewed without hidden network behavior? Can its evaluation framework make missing evidence visible instead of replacing it with plausible numbers? The local evidence supports progress on those engineering questions. It does not answer whether compression improves a particular production application's model quality, cost, or end-to-end latency.

2. Problem formulation and operational definitions

Let x denote an input string and let f_l denote the transformation selected by profile l. The candidate string is y = f_l(x). Let T identify a counting procedure, including its algorithm, model assumptions, and exactness claim. A measured input reduction is then defined relative to T, not relative to an abstract universal token. When T is the built-in estimator, the result is explicitly an estimated reduction. When T is a provider text-tokenization endpoint, the result concerns the submitted text under that endpoint and selected model. Neither definition automatically includes request-envelope overhead or all components of billed inference.

For a nonempty count, the reported reduction percentage is one hundred multiplied by the difference between original and candidate counts, divided by the original count. Empty inputs receive a zero percentage rather than an undefined division. This convention is operationally convenient and avoids manufacturing an infinite improvement from an empty baseline. The implementation also requires nonnegative integer counts and rejects a candidate whose count increases under the same counter. The latter condition protects the advertised local accounting objective; it is not a semantic validator.

We distinguish five properties that are frequently conflated. Byte identity means that the encoded content remains unchanged. Ordered-unit identity means that the same maximal nonwhitespace units occur in the same order. Formatting acceptability means that the application permits the particular spacing change. Task correctness means that the resulting model output satisfies a task-specific evaluator. Semantic equivalence means that the original and optimized prompts retain the relevant meaning under an explicitly stated interpretation. The current release checks the first property for protected prompts and the second property for candidates. It does not claim to decide the remaining properties universally.

A compression result is an evidence object, not merely a replacement string. It contains the original and optimized text, counts, reduction percentage, tokenizer identity, exactness flag, local processing duration, transformation records, warnings, validation outcomes, fallback status, and an explicit indication that semantic equivalence was not assessed. This representation allows application code to impose stricter acceptance policies. An organization might reject every changed prompt until a task evaluator is attached, or permit only a reviewed low-profile transformation for a specific prompt template.

Reversibility also has a precise scope. The package can restore the original input because that input is retained in the returned record. It does not reconstruct missing provider state, reverse an already transmitted request, refund a bill, or undo a downstream action taken by another system. Once an optimized prompt has been used to authorize external effects, local restoration is insufficient as a transaction rollback. Integrators must therefore place transformation approval before consequential model-driven actions and preserve their own application-level control boundaries.

The optimization objective is intentionally subordinate to constraints. Given a prompt and policy, the software seeks a candidate with a nonincreasing count while preserving required mechanical invariants. If those constraints fail, the original string is returned. If recognized protection rules apply, transformation is skipped before candidate generation. These two cases should remain distinguishable: a protected identity result is an expected policy outcome, whereas fallback records a candidate that failed validation. Combining them into a single failure statistic would obscure both safety behavior and engineering defects.

3. Related work and positioning

Learned prompt compression addresses a broader problem than the formatting rules implemented here. LLMLingua introduces a coarse-to-fine approach involving budget control, iterative token selection, and alignment between language models [1]. LLMLingua-2 frames compression as token classification using distilled supervision and an encoder-based architecture [2]. These research programs examine how substantially shorter representations can retain useful task information. TokenPropulsion does not reproduce their algorithms, claim their compression ratios, or transfer their published quality results to its own software. No comparative execution of either project was performed for this release.

The distinction matters when interpreting the word compression. A learned system may remove lexical content using a model-dependent estimate of importance. TokenPropulsion's implemented profiles retain all nonwhitespace units and operate only on a limited family of formatting changes. Its likely reduction ceiling is therefore much lower on ordinary prompts. This limitation is deliberate. The project prioritizes a transparent integration boundary and reproducible mechanical behavior before introducing transformations whose acceptance would depend on learned judgments about relevance. Future extensions must be evaluated as new methods rather than inheriting safety claims from the present release.

Research on long-context models also cautions against interpreting input length as an adequate description of task difficulty. Lost in the Middle studies how the position of relevant information influences retrieval and question-answering performance [3]. Its relevance here is methodological: an evaluation should vary information location and task structure, rather than measuring only the number of characters removed. TokenPropulsion currently preserves content order. That property avoids deliberate reordering, but it does not establish that changing blank-line boundaries leaves a model's attention or behavior unchanged.

Instruction-hierarchy research examines the distinction between privileged instructions and lower-trust material [4]. For preprocessing systems, the corresponding engineering obligation is to avoid promoting quoted content, retrieved passages, or user-controlled text into authoritative instructions. TokenPropulsion's adapter sends prompt material as a user message and does not rewrite it into a system or developer role. This is a limited boundary-preservation measure. It neither trains the downstream model to follow a hierarchy nor constitutes a general defense against prompt injection. Such claims would require a separate adversarial model evaluation.

Subword tokenization research provides another reason to avoid equating characters, words, and model tokens. Sennrich, Haddow, and Birch describe subword units in neural translation [5]. The immediate implication for this project is conceptual: token boundaries depend on a specified representation procedure. An arithmetic rule over bytes cannot be treated as an exact implementation of an arbitrary model's vocabulary. The built-in estimator is therefore presented as an explicit heuristic, while the optional provider tokenizer records its narrower text-tokenization scope.

This positioning leaves a useful but constrained contribution. TokenPropulsion provides an inspectable foundation for applications that need conservative formatting, accounting provenance, reversible results, and a reproducible evaluation harness. It is not a replacement for learned compression research, a tokenizer

certification service, or a semantic-equivalence theorem prover. A future comparison with established projects should evaluate task quality and complete operating cost under matched conditions, while preserving the original projects' licenses and clearly identifying all additional model dependencies.

4. Architecture and dependency boundaries

The package follows a source-layout Python architecture. The public import exposes compress_prompt and CompressionResult, while separate modules implement profiles, input security, protection detection, validation, counting, metrics, and errors. This decomposition is intended to make the trust boundaries inspectable. A reviewer can examine transformation rules independently of provider credentials, and an application can import the core without importing FastAPI or the official xAI SDK. Optional integrations occupy dedicated namespaces rather than becoming hidden requirements of basic string processing.

The core requires only the Python standard library. This decision reduces installation complexity, limits the default dependency graph, and makes offline behavior easier to verify. It does not imply that standard-library code is automatically secure or that optional dependencies are unimportant. The optional API and provider adapters have their own dependency surfaces, supported version constraints, security advisories, and behavior changes. The repository consequently records a tested environment snapshot and includes a dependency-auditing workflow. That snapshot describes a particular validation environment; it is not represented as a portable lock covering every supported interpreter.

The processing path begins with input validation. Non-string inputs, invalid Unicode scalar sequences, NUL characters, and inputs above the configured byte limit are rejected. A protection pass then examines the complete prompt. If any recognized risky pattern is found, the candidate remains identical to the input. Otherwise, the selected profile applies deterministic formatting operations in a fixed sequence. Each changed intermediate value becomes a transformation record. Independent validators inspect the final candidate, and the same counting interface measures both original and candidate strings.

Results use frozen dataclasses to discourage accidental mutation of audit fields after construction. The immutability contract concerns the package's result records; it does not prevent callers from copying text, modifying exported dictionaries, or retaining sensitive content elsewhere. The to_dict method creates an ordinary serialization-friendly representation. Application developers must recognize that exporting this structure includes original prompts and transformation snapshots. The convenience of a complete audit trail must be balanced against retention policies, access controls, and the possibility of sensitive input appearing in diagnostic artifacts.

Failure behavior is divided into several classes. Invalid application input raises a stable input exception. Disabled network access raises a dedicated authorization exception. Provider failures are sanitized to avoid including credentials or private prompt content. Candidate validation failure returns an original-text fallback with warnings and validation outcomes. This separation helps callers decide whether to correct a request, change policy, retry infrastructure, or accept an unchanged prompt. A single generic success flag would conceal those materially different operational situations.

The CLI, HTTP service, and benchmark framework are consumers of the same core API. They do not reimplement compression rules. This reduces divergence between documented behavior and integration behavior, although each interface still needs its own tests for encoding, limits, serialization, and error handling. The architecture favors explicit boundaries over feature abundance: core processing, external communication, persistent export, and publication remain separate actions whose evidence can be inspected independently.

5. Protection policy and refusal behavior

Protection is applied at whole-prompt granularity. Recognized code fences, inline-code markers, quoted literals, common structured-data punctuation, addresses, paths, identifiers, numerical expressions, critical instructions, and sensitive markers prevent every formatting transformation. Some Unicode ranges and control characters also trigger preservation. This approach deliberately accepts false positives. A harmless sentence containing a colon may remain unchanged even when a human could safely normalize it. The policy trades potential reduction for a smaller and more inspectable set of transformation opportunities.

Whole-prompt preservation avoids a class of errors introduced by partial masking. A masking implementation must recognize exact spans, choose placeholders that cannot collide with user text, preserve nesting, and restore original bytes in the correct order. Malformed fences, overlapping syntax, and adversarial placeholder strings complicate that process. TokenPropulsion avoids those particular restoration hazards by declining the entire transformation when it recognizes protected material. This choice does not solve language recognition in general; it reduces the amount of parsing confidence required before refusing a change.

The policy is heuristic rather than grammatical. It does not implement complete parsers for Markdown, Python, SQL, JSON, YAML, XML, or every configuration language. A plain sentence can also be a valid scalar in a structured format, and an unmarked fragment can carry application-specific syntax that no general detector recognizes. Consequently, absence of a protection match must never be interpreted as proof of safe prose. Applications with stronger requirements should supply domain-specific admission rules or reject all changed prompts until their own evaluator approves them.

Whitespace examples demonstrate why this caution is necessary. CommonMark assigns meaning to certain spaces preceding a line break [6]. JSON string contents can contain spaces whose removal changes the value even when surrounding structural whitespace is insignificant [7]. Indentation can determine the structure of source code or configuration data. The implementation therefore protects recognized Markdown hard breaks, quoted strings, tabs, and common indentation patterns instead of assuming that a regular expression over whitespace is universally harmless. These cases are covered by targeted regression tests.

Critical instructions receive similarly conservative treatment. Negations, exact-output language, system or developer terminology, security-policy references, and logical operators can be decisive to task behavior. The detector recognizes selected English and Romanian forms, but it is not a multilingual security classifier. Recognized terms freeze the prompt; unrecognized paraphrases may not. The published policy is therefore a visible limitation of the release, not a complete taxonomy of sensitive language. Expansion of this detector requires regression evidence and careful consideration of false-positive behavior.

A refusal should be operationally ordinary. When a prompt is protected, its optimized text equals its original text, its transformation list is empty, and its warnings explain the protection categories. Users can inspect that outcome without treating it as a malfunction. Benchmark reports should count protected identity results separately from validator-triggered fallback. In safety-oriented preprocessing, a zero-reduction result can be the correct result. A system that pressures every input into a shorter representation creates incentives to weaken precisely the constraints that justify using it.

6. Transformation profiles and mechanical validation

The low profile removes eligible trailing ASCII spaces. The medium profile adds normalization of runs containing excessive blank lines. The high profile additionally collapses repeated ordinary spaces in admitted prose. High mode is explicitly opt-in and includes an additional warning. None of these profiles deletes words, merges instructions, summarizes passages, changes message roles, or removes repeated sentences. The labels describe increasing formatting intervention, not increasing demonstrated semantic compression. This terminology is documented to prevent users from inferring capabilities that the implementation does not provide.

Operations run in a fixed order, and each actual change records its name, before value, and after value. Recording intermediate values makes the transformation path inspectable without asking users to infer the responsible rule from a final diff. It also supports precise regression tests: an application can assert which operations were applied and compare the final candidate with an approved fixture. The cost is additional memory proportional to the number of changed stages. The input-size limit bounds ordinary exposure, but applications still need concurrency controls when processing many requests simultaneously.

The principal textual validator extracts maximal nonwhitespace units from the original and candidate and requires identical sequences. This catches deleted words, changed punctuation attached to words, substitutions, and reordering of such units. A second validator requires byte identity when protection applies. A third checks that the candidate count does not exceed the original count under the selected counting procedure. These checks are independent of the transformation implementation, which is valuable for detecting accidental defects introduced by later rule changes.

A conditional proposition can be stated precisely. If the ordered-unit validator passes, then the sequence produced by that extraction procedure is identical for the original and candidate. The proposition follows directly from the comparison performed by the implementation. It says nothing about the meaning of spaces, paragraph boundaries, or layout in the originating application. Extending this proposition into a semantic-equivalence claim would require an interpretation of those features and evidence that the transformation preserves it. The release intentionally does not make that extension.

Fallback occurs whenever a required validation result is negative. The software then returns the original string, restores the original count, clears the applied transformation list, and preserves warnings and validation outcomes indicating why the candidate was rejected. Clearing the applied list describes the returned result rather than the discarded candidate; the validation records remain evidence of the attempted operation. Applications that need complete rejected-candidate retention should design an explicit privacy-conscious extension rather than assuming the current result stores every unsuccessful intermediate proposal.

Tokenizers are also part of the failure boundary. The package rejects invalid negative or noninteger counts, and it does not silently replace a failing custom counter with the built-in estimate. Silent substitution would change the meaning of the accounting result without caller authorization. A custom counting implementation therefore remains a trusted integration component whose identity and exactness claims must be reviewed by its owner. The protocol enables extensibility, but extensibility cannot certify the correctness of arbitrary implementations supplied by callers.

7. Token accounting and provenance

The built-in counter computes a configurable estimate from the UTF-8 byte length of the input, rounding upward after division by a positive finite scale. Its default scale is four bytes per estimated token. That default is an operational heuristic, not an empirical calibration for every language or a claim about a specific provider vocabulary. The result identifies the procedure by name and marks its counts as estimated. The estimator's exactness field cannot be switched to true through ordinary constructor configuration. This prevents a simple parameter mistake from relabeling the built-in arithmetic heuristic as an official model tokenizer.

Byte length is an attractive dependency-free signal because it is deterministic and available offline. Its shortcomings are equally important. Different scripts encode into different byte patterns, linguistic units vary, and actual tokenizers may combine or split whitespace in model-specific ways. Even within one language, code, punctuation, repeated spaces, and uncommon identifiers can produce different tokenization behavior. The estimate is neither a universal upper bound nor a confidence interval. Users should not interpret a positive estimated reduction as proof that their provider will consume fewer billed input tokens.

The optional xAI tokenizer calls the documented text-tokenization interface and counts the returned token sequence [8]. Its identity includes the provider and selected model. This is a more specific accounting procedure than the byte estimator, but its scope still matters: a text-tokenization call describes submitted text, whereas a complete inference request may include role serialization, internal overhead, cached components, or other provider-defined accounting. The adapter separately exposes usage returned with actual inference responses. These two measurements should be reported under their own names rather than combined into one apparently exact count.

A further distinction concerns measurement placement. Calling a remote tokenizer from inside compression makes the measured processing interval include network activity. Such a result cannot be compared directly with a purely local preprocessing benchmark while preserving the interpretation of local CPU overhead. The offline runner consequently fixes its accounting to the offline estimator. Applications may supply a remote counter to the public API, but must then interpret the returned duration as the complete operation performed by that configuration, including those counting calls.

Pricing follows the same provenance principle. The package contains no built-in claim that a particular model currently costs a particular amount. A caller supplies model-specific input pricing together with currency, source location, and verification date. The comparison helper can calculate an input-cost difference under that supplied rate. It does not infer cached-token discounts, output-token charges, negotiated prices, taxes, or credits. Accurate economic evaluation therefore requires both trustworthy usage and a pricing model appropriate to the account and request type.

The practical implication is that accounting should travel with its assumptions. A useful report states which tokenizer or estimator ran, whether it was exact for its declared scope, which model was selected, when pricing was verified, and whether counts came from text tokenization or completed inference. Numbers without that context are difficult to reproduce and easy to overinterpret. TokenPropulsion treats these labels as part of the measurement, not as optional explanatory text added after a savings claim has already been made.

8. Cost and latency models

A preprocessing system can improve input size while worsening end-to-end latency. Let L_c describe local compression time, L_o describe the original request latency, and L_y describe the optimized request latency under a matched model configuration. The optimized path is faster only when L_c + L_y is less than L_o under the deployment conditions being evaluated. A token-reduction percentage alone cannot decide that inequality. Network variation, server load, prompt caching, output length, and model scheduling can dominate small differences in input processing.

The same caution applies to monetary cost. Under a simple fixed input rate, the nominal input difference is proportional to the difference in billed input tokens. However, an actual experiment may change output length or cache eligibility, and retries can duplicate billable work. A useful economic model includes preprocessing infrastructure, input and output billing, failed requests, repeated attempts, storage, and operational support. A local library with no required dependencies may have low acquisition cost, but its deployment and evaluation still consume engineering time and computing resources.

TokenPropulsion records local processing separately from remote request duration. The remote runner includes original and optimized request times, paired remote duration, compression time, and per-variant end-to-end fields. This permits several comparisons without pretending they are interchangeable. A paired duration describes the cost of conducting an experiment with two calls. A per-request duration describes one provider interaction. An end-to-end optimized duration adds the measured preprocessing cost. Applications should choose the statistic that corresponds to their actual execution path rather than selecting whichever field appears most favorable.

Latency distributions also require care. The offline report computes p50 and p95 by linear interpolation over sorted observations. These percentiles summarize the observed sample, not a guaranteed service level. The synthetic dataset mixes short prompts with a substantially longer repeated document, so aggregate tails partly reflect input composition. A production study should report length-stratified distributions and enough repetitions to characterize variability. It should also record hardware, interpreter, concurrent load, warm-up policy, and whether timing includes initialization or dependency import.

The remote experiment alternates original-first and optimized-first ordering across repetitions. This reduces one obvious ordering confound but does not establish randomized experimental control. Cached requests, temporal provider variation, and correlation between adjacent calls may remain. A stronger study would randomize or block order, document cache policy, preserve model identifiers, and report uncertainty over task-level paired differences. The present runner provides a repeatable starting protocol; it does not convert a small repeated sample into evidence of universal acceleration.

Failed remote pairs create another reporting hazard. Some requests may complete and incur charges before a later call or evaluator fails. The runner records sanitized failures, but successful-pair summaries do not reconstruct every partial bill. Users must therefore reconcile experiments with provider usage records when computing total spend. Excluding failed observations from latency summaries can also bias results toward easier or faster requests. A trustworthy report states that selection effect instead of presenting successful-only measurements as unconditional production performance.

9. xAI adapter and explicit external communication

The optional adapter uses the official xAI Python SDK rather than a reimplementation of its transport protocol. Synchronous and asynchronous request methods share the same model selection and generation configuration. Each request constructs a chat interaction, appends the supplied text as a user message, disables stored chat messages through the documented request option, and samples a response. The adapter extracts provider-returned prompt, completion, and total usage fields together with response content and elapsed remote time. The installed SDK's signatures and usage structures are checked offline in integration tests.

External communication requires explicit authorization. Creating the adapter with its default configuration does not authorize inference. Request methods verify the network flag before consulting credentials or contacting the provider. The API key is read from the process environment, and missing credentials cause a sanitized configuration error. A separate authorization flag is required when recognized sensitive-content markers are present. These controls make accidental transmission harder, but marker detection remains heuristic. Applications must review their datasets and provider policies independently before sending confidential information.

Timeouts and retries are bounded configuration choices. The adapter permits a finite per-attempt timeout and a limited retry count. Transient gRPC statuses such as resource exhaustion or temporary unavailability may be retried when the caller enables retries. The default retry count is zero because an ambiguous inference failure can already have incurred a charge. Backoff is bounded, and authentication or ordinary application failures are not treated as automatically retryable. This policy provides predictable limits without implying that every retry is economically or semantically harmless.

Provider errors are intentionally sanitized. The public exception does not include raw server details, credentials, or prompt content. This reduces a common route by which private information enters application logs. It also reduces diagnostic detail, so integrators may need controlled internal observability appropriate to their security requirements. Such observability should distinguish operational metadata from prompt bodies and should not disable the package's privacy contract by indiscriminately printing exception objects or complete request records.

The adapter's compare method performs two billable requests: one for the original text and one for the optimized text. Its name describes a request comparison, not an automatic semantic judgment. The caller receives both responses and the compression record, then applies an evaluator suited to the task. Identical temperature and output limits improve comparability, but output variability and model-specific parameter support still require live verification. A model rejecting a parameter is an integration failure to report, not evidence that the compression method itself improved or degraded quality.

No live xAI inference or tokenization was executed in the recorded local validation because no API key was configured. Mocked transport behavior establishes that adapter code calls the expected interfaces and handles simulated outcomes. It cannot establish account access, current model availability, real provider retention, production latency, billing correctness, or live retry behavior. Those limitations remain explicit in the README, validation report, and release notes. Publishing an adapter is therefore separate from claiming externally verified interoperability.

10. CLI, encoding, and audit export

The command-line interface serves both interactive use and pipelines. Positional text, file input, and standard input feed the same compression API. Optimized text is written to standard output, while optional statistics are written to standard error. This separation is essential for composability: a downstream program expecting prompt text must not receive a mixture of prose and diagnostic JSON. Machine-readable output is explicitly requested through JSON reporting or the analyze command. Error exits distinguish ordinary success from invalid input and broken-pipe behavior.

Encoding is part of the correctness boundary. The CLI reads bounded UTF-8 bytes from files and binary standard input when available, then writes UTF-8 output without platform newline translation. Protected CRLF content must remain byte-identical across file and pipeline round trips. This is particularly important on Windows, where convenient text-mode defaults can otherwise normalize line endings even when the compression engine itself returns an unchanged string. Regression tests and fresh-install smoke checks exercise that behavior through the installed executable, not merely through a direct Python function call.

JSON reports deliberately expose more information than plain optimized output. They include original text and stepwise transformation records so a caller can inspect and reverse the operation. That feature also makes an exported report potentially sensitive. The package does not silently save it, but a user choosing an output path or redirecting JSON has created a persistent prompt-bearing artifact. Documentation therefore distinguishes default stateless operation from explicit export. Organizations should apply retention and access policies to these reports as they would to other application input records.

Statistics mode avoids copying original or optimized text into diagnostic output. It also omits transformation snapshots, which could otherwise reintroduce the same private content through an indirect field. Warnings describe recognized protection categories and validation status rather than quoting prompt fragments. This is a practical privacy property of the CLI design. It does not guarantee that every downstream shell history, process listing, terminal recording, or wrapper script is private. Passing sensitive text as a positional command-line argument can expose it outside the package's logging behavior.

The documented interface reserves analyze and benchmark as positional commands. A literal prompt equal to one of those command names should therefore be supplied through a file or standard input. This is a visible usability constraint, not an undocumented parser surprise. File errors and invalid datasets return concise sanitized messages. The CLI does not automatically submit benchmark prompts to a model endpoint, and its benchmark command remains offline. Remote experiments require an explicit Python invocation with an authorized adapter and an evaluator.

These details illustrate a broader lesson: a correct transformation function can still be wrapped by an incorrect interface. Newline conversion, output-channel mixing, unbounded reads, accidental serialization, and ambiguous commands can invalidate an otherwise careful core. Integration tests should therefore follow information from the input boundary to the actual output stream. TokenPropulsion treats those tests as behavioral evidence rather than relying exclusively on isolated unit coverage.

11. HTTP service and deployment boundary

The optional FastAPI service exposes compression, analysis, health, and version endpoints through an application factory. Startup requires a configured ASCII bearer token of at least thirty-two characters. Documentation and schema endpoints are disabled by default, and every endpoint requires authorization. The documented development command binds to loopback. These choices establish a conservative starting configuration, but they are not a complete public deployment solution. Operators remain responsible for transport security, rate limits, concurrency limits, access-log policy, request timeouts, and secret management.

Request-body size is checked while reading the incoming stream and before JSON parsing. The service rejects a chunk when accepting it would exceed the configured encoded-body limit, avoiding an additional oversized accumulation. The limit applies to the entire request body, not merely the decoded text field. Unsupported content encodings are rejected rather than automatically decompressed. This prevents a compressed payload from bypassing the intended accounting boundary. A proxy and server should still enforce their own limits because application middleware cannot eliminate every transport-level resource exhaustion scenario.

Input models enforce permitted fields, string types, profile names, and character limits. The core additionally validates UTF-8 byte size and unsupported input characters. These layers address different representations: a character count does not necessarily equal an encoded byte count. Custom validation-error handling avoids returning raw submitted values in error responses. This matters because framework defaults may otherwise echo input fragments during diagnostics. The service returns concise failure categories without retaining a prompt history or adding a telemetry system.

Authorization is checked before normal request processing. Constant-time comparison is used for the bearer credential, and no fallback development token is embedded in source code. The token must still be generated with sufficient entropy by the operator; a length check cannot prove randomness. Deployments should obtain secrets from an appropriate secret manager or environment provisioning system and rotate them according to their own policy. Documentation examples identify placeholders as placeholders rather than distributing a shared credential that could accidentally become a production default.

Statelessness limits one privacy risk but does not remove all operational data exposure. A reverse proxy may log request metadata, a monitoring agent may inspect memory, and a caller may save returned JSON. The API response itself contains the original and optimized text because it exposes the audit result. Consequently, transport encryption and caller-side retention are important even though the application writes no prompt database. Public deployment should be reviewed as a system, including infrastructure that sits outside this repository.

The local tests verify configured authentication, endpoint behavior, strict input validation, sanitized errors, unsupported encoding rejection, and oversized requests. They do not constitute a penetration test, formal verification of the HTTP stack, or load qualification at a particular concurrency level. The service is an optional integration component suitable for further evaluation. Treating the existence of authenticated endpoints as proof of production readiness would exceed the evidence provided by the executed suite.

12. Offline dataset and baseline design

The checked-in dataset is synthetic and intentionally small. Its nine categories cover repetitive instructions, coding requests, structured outputs, retrieval-style context, multilingual text, long-form documents, financial or numerical tasks, exact-match instructions, and adversarial edge cases. Each row has an identifier, a category, input text, and a deterministic expected optimized value. These fixtures test the documented formatting and protection contract. They are not a representative sample of commercial workloads, and they do not support population-level estimates of compression performance.

Three accounting views appear in the offline report. The unchanged-input baseline records what happens without compression. A deliberately simple whitespace-normalization baseline illustrates the amount of estimated reduction available when protection is ignored. The implemented method applies its protection and validation policy. The naive baseline is not a recommended optimizer and is never automatically sent to a provider. Its role is analytical: it demonstrates why the apparent opportunity for shorter text must be considered together with the risk of changing structured or exact-output content.

The long-form fixture repeats a simple prose pattern many times. This makes it useful for exercising a longer input and observing scaling, but it also gives that row substantial influence over aggregate estimated counts. A headline percentage over the complete dataset would therefore be sensitive to this construction choice. Reports should present category-level or row-level outcomes alongside totals. A future evaluation should add realistic length distributions, mixed syntax, difficult boundary cases, and reviewed application templates before making claims about expected savings in practice.

The multilingual fixture includes Romanian text, while security tests cover additional scripts and Unicode edge cases. This demonstrates particular behaviors of the implementation; it is not multilingual semantic validation. Recognized unfamiliar scripts are preserved, which avoids modifying those samples but does not measure compression quality for them. A comprehensive multilingual study would require native-language task evaluators, script-specific tokenizer analysis, and examples whose meaning depends on punctuation, spacing, or orthographic conventions. Zero changes on protected text should remain a valid reported outcome.

Dataset hashes identify the precise bytes used for a run. The offline report also records profile, repeat count, estimator identity, exactness, row-level accounting, timing percentiles, fallback rate, fixture outcomes, and explicit missing remote metrics. Hashing does not certify dataset quality or remove sampling bias. It makes disagreements easier to investigate by distinguishing different input versions from different execution behavior. Users can reproduce the content and deterministic results while expecting wall-clock measurements to vary across machines and loads.

The dataset is published under the project's license because it was created for the project and contains no customer prompts. That origin simplifies redistribution and review, but synthetic safety is not automatic realism. The benchmark should be understood as a regression and measurement scaffold. Its scientific value comes from exposing the procedure and limitations, not from the size of a selected reduction figure. Expanding it responsibly requires documented provenance, privacy review, and task-specific correctness criteria.

13. Recorded local results and their interpretation

The recorded validation environment used Windows x64 and Python 3.12.14. The complete local suite passed 191 tests with zero failures, errors, or skips. The measured line coverage was 99.61 percent, with 509 of 511 counted statements covered. Combined line-and-branch coverage was 99.54 percent. The uncovered entry-point statements were separately exercised through installation smoke checks, although those subprocess executions were not merged into the coverage measurement. High coverage indicates exercised code paths; it does not prove that every meaningful behavior or failure mode has been specified.

Ruff checking and formatting validation passed, and strict mypy reported no issues across twenty package modules. The source distribution and wheel built successfully, including a wheel build from the source distribution. A fresh virtual environment installed the wheel without optional runtime dependencies. Installed-command smoke checks verified the Python API, help output, separation of text and statistics, and CRLF preservation through both pipelines and files. These checks address release engineering and interface behavior rather than downstream model quality.

The offline benchmark's unchanged inputs totaled 3,829 estimated tokens, while protected formatting produced 3,578 under the configured estimator. The aggregate estimated reduction was approximately 6.56 percent. Every deterministic fixture passed, and validator-triggered fallback was zero for this dataset. These are measurements of the published synthetic inputs and the declared arithmetic procedure. They must not be restated as actual xAI token savings, provider cost reduction, task success, or evidence that the original and optimized prompts mean the same thing.

The checked-in run reported a local p50 of approximately 0.0615 milliseconds and p95 of approximately 16.0 milliseconds across repeated observations. The large separation reflects the mixed input lengths and local execution conditions. These values are descriptive observations from that report, not a service guarantee or a claim of superiority over another optimizer. Reproduction can change timings even when deterministic text and count outcomes remain identical. No benchmark against LLMLingua or another established compressor was executed.

Security checks found no Bandit source findings and no high-confidence secrets under the bounded repository scan. The dependency audit reported no known vulnerabilities in the resolved audited dependencies; the unpublished local distribution itself was skipped because it had no public advisory record. These outcomes are time-sensitive and limited by scanner coverage. They are not certifications, evidence of exhaustive secret absence, or substitutes for independent review. The repository preserves the actual machine-readable reports so reviewers can inspect what was checked.

One non-failing warning arose from Starlette's deprecation of its httpx test transport in favor of httpx2. The warning was retained rather than suppressed. Local integration tests still passed. The supported-Python GitHub matrix had not yet completed in the original local evidence set, and no live provider request was made. Those boundaries explain the release-candidate designation. A responsible first public release should preserve that designation until the relevant external checks and compatibility evidence justify a stronger statement.

14. Output evaluation and semantic claims

A useful evaluation distinguishes the optimizer's mechanical contract from the downstream task's success criteria. A formatting fixture can show that a recognized code block remained unchanged or that an admitted prose template lost redundant blank lines. It cannot show that a language model will produce an equivalent answer. Conversely, a correct model answer on one trial does not prove that the transformed prompt preserved every relevant constraint. The evaluation must specify what success means, which constraints matter, and what kinds of failure the chosen metric can detect.

Exact-match tasks offer a strong but narrow evaluator. If the application requires an exact string, equality provides a deterministic decision. The package includes this evaluator because it is inspectable and does not require another model to judge the result. Its limitations are straightforward: acceptable paraphrases may be rejected, while a correct short answer may conceal unsupported reasoning or omitted evidence when those properties matter. Exact matching is appropriate only when the task's definition actually supports it. It should not be adopted merely because it produces a convenient binary score.

Structured-output evaluation requires attention to parser behavior. The JSON evaluator canonicalizes parsed values, distinguishes booleans from numerical values, rejects nonfinite numerical constants during serialization, and rejects duplicate object keys. These decisions avoid several false positives that arise from ordinary host-language equality or permissive parsing. RFC 8259 supplies the relevant data-interchange context [7]. The evaluator remains an application choice: semantically acceptable numeric tolerance, ordering requirements, and domain-specific constraints may need additional logic beyond structural comparison.

The included schema helper supports an explicitly limited subset covering declared types, object properties, required keys, and additional-property restrictions. Unsupported keywords are rejected rather than silently ignored. It is not advertised as a complete JSON Schema implementation. A production task requiring ranges, patterns, array-item constraints, or complex schema composition should attach a separately reviewed validator. Narrow functionality with explicit rejection is preferable to a broad-looking interface that quietly fails to enforce part of the declared contract.

For open-ended outputs, evaluators should be designed before comparing compression methods. A rubric may assess factual correctness, constraint adherence, citation support, completeness, and harmful deviations. Human review, deterministic checks, and model-assisted judgments can complement each other, but each introduces assumptions and potential bias. If another model serves as judge, its version, prompt, sampling policy, and agreement with independent reviewers should be documented. TokenPropulsion does not silently insert such a judge or present its hypothetical judgments as measured semantic equivalence.

The release therefore exposes an explicit false value for semantic-equivalence assessment. This field is not an admission that every changed prompt is wrong. It is a statement about the evidence actually collected. An application may add task validation and stronger acceptance criteria, but it should preserve the distinction between that application-specific result and a universal semantic guarantee. Scientific credibility depends on keeping the scope of the conclusion aligned with the scope of the test.

15. Experimental protocol for future remote studies

A defensible remote study begins with a written protocol. Investigators should identify the application population, choose representative prompts, define protected content, establish task evaluators, and specify the model configuration before inspecting favorable results. The dataset should include both expected optimization opportunities and prompts that should remain unchanged. Excluding difficult or protected cases after observing outcomes would inflate apparent benefit and weaken the interpretation of safety behavior. Public reporting should describe inclusion criteria and record all failures attributable to the experiment.

The principal comparison should be paired at the task level. Each task supplies an original prompt and the candidate generated by a fixed optimizer version. Both requests should use the same supported model, output limits, sampling settings, and relevant account configuration. Repetition is necessary when outputs or infrastructure vary. Pair order can be randomized or balanced within blocks, and cache behavior should be documented. A study should avoid comparing a warm optimized request with a cold original request while attributing the difference entirely to compression.

Primary endpoints should be selected in advance. Examples include exact task success, valid structured output with domain constraints, input billing difference under verified pricing, and optimized-path end-to-end latency. Secondary endpoints might include output length, failure categories, protection rate, fallback rate, and evaluator disagreement. A single composite efficiency score can obscure tradeoffs, particularly if a modest cost improvement is allowed to compensate numerically for a serious correctness failure. Reporting the underlying dimensions separately makes those tradeoffs inspectable.

Uncertainty should be computed over the right unit. Repeating one synthetic prompt many times does not create many independent application tasks. A suitable analysis can summarize paired task-level differences and provide intervals obtained through an appropriate resampling or statistical method. The protocol should explain how failures, timeouts, and invalid outputs enter the analysis. Reporting only completed pairs can bias latency and quality estimates. Failed attempts may still consume time and money, so exclusion rules must be justified rather than silently applied.

Privacy authorization is another protocol requirement. Even a research dataset can contain personal information, credentials, licensed text, or confidential business context. The adapter's sensitive-marker guard is not a substitute for human review and organizational authorization. Public reproducibility may require a sanitized synthetic counterpart or an access-controlled evaluation package. Researchers should separate the ability to reproduce a method from an obligation to publish every original prompt. Hashes and aggregate reports help, but they do not automatically make private datasets safe to disclose.

The current remote runner supports a limited version of this paired workflow with repeated measurements, alternating order, caller-supplied evaluators, returned usage, and sanitized failure records. It is infrastructure for a future study, not a completed experiment. Before making a production claim, investigators should extend it where their protocol requires randomization, account-level billing reconciliation, evaluator auditing, or stronger provenance. The most important next result would be an independently reproducible application study, including cases where preprocessing provides no benefit.

16. Security model and residual risk

The threat model includes accidental corruption, misleading accounting, unintended network transmission, credential leakage through errors, prompt retention through diagnostics, and resource exhaustion at exposed interfaces. It also includes adversarial inputs intended to cross trust boundaries or exploit incomplete syntax recognition. TokenPropulsion reduces selected risks through offline defaults, whole-prompt preservation, bounded input sizes, sanitized exceptions, explicit export, and authenticated optional endpoints. These controls are concrete implementation measures. They do not imply that the project is a complete security boundary for an autonomous agent or a hostile multi-tenant deployment.

An important residual risk is false admission. The detector may fail to recognize a prompt whose whitespace has application-specific meaning. Mechanical validators can then pass even though the application's intended interpretation changes. The appropriate response is layered policy: restrict admitted templates, protect entire classes of sensitive prompts, evaluate downstream tasks, and retain a bypass mechanism. An application with strict exactness requirements may reasonably disable optimization altogether. The library should make that decision easy rather than presenting refusal as a failure of adoption.

Another risk arises from audit richness. Keeping the original text makes restoration straightforward, while keeping intermediate snapshots improves inspection. Those same fields can multiply sensitive material in memory or exported files. A caller that sends complete result dictionaries to a logging platform has created a retention channel even though the core itself is stateless. Integrators should design explicit redaction or metadata-only telemetry and document which fields leave the process. The default CLI statistics path demonstrates one limited approach by omitting text-bearing fields.

External services add independent boundaries. Disabling a stored-message option is not equivalent to proving that a provider retains no operational data. SDK telemetry, network infrastructure, account settings, and legal obligations may affect the full data lifecycle. This whitepaper does not certify any provider's privacy policy. Organizations should evaluate the official documentation and their agreements before transmitting confidential prompts. The adapter's role is to make communication explicit and avoid adding undisclosed compression calls, not to replace that institutional assessment.

Supply-chain risks persist even when the core is small. Optional dependencies, build backends, test tools, and workflow actions execute code during installation or automation. The repository pins third-party workflow actions to immutable commits, uses restricted permissions, and runs source and dependency checks. Those controls improve traceability but still require maintenance and review. A pinned vulnerable action remains vulnerable, and a clean advisory result can become outdated. Publication governance should therefore include updates, incident handling, and review of newly introduced dependencies.

The bounded secret scanner searches selected high-confidence patterns without printing matched credentials. Its exclusions and pattern coverage mean that a zero-findings result is not a proof of exhaustive absence. Human review of staged files remains necessary, especially before the first public push or release attachment. No private customer prompt was included in the published synthetic fixtures. That is a dataset-provenance statement, distinct from any guarantee that a future contributor cannot accidentally add sensitive material.

17. Testing strategy and evidence quality

The automated suite combines targeted unit tests, interface integration tests, adversarial protection cases, and deterministic randomized regression checks. Unit tests cover profiles, counting, restoration, validation, pricing arithmetic, percentile behavior, and malformed inputs. Integration tests cover files, pipelines, JSON output, the authenticated API, and mocked SDK requests. Security cases exercise code syntax, quoted literals, paths, numerical expressions, critical negation, sensitive markers, and additional scripts. This diversity is more informative than a large number of tests that merely repeat the implementation's internal structure.

Deterministic randomized tests generate synthetic combinations of words, markers, and whitespace under a fixed seed. They verify that ordered nonwhitespace units remain identical, restoration returns the original text, and optimized counts do not increase. Such tests can explore interactions that individual fixtures miss. They remain limited by the generator's vocabulary and distributions. A fixed seed supports reproducibility, while broader fuzzing campaigns would need separate resource limits, corpus retention policies, and a process for turning discovered failures into readable regression cases.

Long-input tests address a different class of defect. Regular expressions that appear harmless on short samples can exhibit poor behavior when patterns span repeated lines or ambiguous whitespace. The implementation avoids a newline-spanning leading-whitespace pattern that could repeatedly scan long blank input. Regression cases include very long prose and blank-line sequences. These tests do not prove a complete asymptotic complexity bound for every future pattern, but they document the failure mode and make accidental reintroduction easier to detect during maintenance.

Coverage is interpreted as a diagnostic measure. The recorded percentage exceeds the project's target, yet the more important question is whether assertions correspond to meaningful behavior. Tests that assert restoration, protected identity, bounded retries, sanitized errors, and stream separation support concrete contracts. Coverage alone cannot establish adequacy of the protection taxonomy, independence of evaluators, or representativeness of the benchmark dataset. A reviewer should examine test intent and missing threat scenarios rather than treating a badge as a substitute for that inspection.

The supported interpreter matrix is configured in GitHub Actions, while the initial executed local evidence comes from one Python version and operating system. Parsing source under an older grammar checks syntax compatibility only. Runtime semantics, dependency resolution, transport behavior, and packaging can differ across environments. Hosted matrix outcomes should therefore be treated as additional evidence when available, with their exact commit and workflow run recorded. A first release must not describe unexecuted matrix jobs as passing merely because the configuration file exists.

Environmental failures also require honest classification. The Windows sandbox prevented a local asyncio socketpair and blocked some default temporary-directory operations. Running the offline and mocked tests with appropriate local permissions and a workspace temporary directory resolved those failures. This observation should neither hide a real product defect nor be misreported as a successful test before execution. The validation record explains the environment adjustment so another reviewer can distinguish infrastructure restrictions from changes to the software's acceptance criteria.

18. Reproducibility and release governance

Reproducibility has several layers. Source reproducibility requires a specific commit and licensed code. Data reproducibility requires the dataset bytes or an appropriate access mechanism. Environment reproducibility requires interpreter and dependency versions. Measurement reproducibility requires command lines, configuration, repetition policy, and timing definitions. Publication reproducibility requires identifiable artifacts and checksums. TokenPropulsion's repository provides these ingredients where they were available, while explicitly leaving external provider outcomes unmeasured. No single hash or build badge can replace the complete chain.

The release-candidate version communicates incomplete external validation. It does not mean that local build or test failures are acceptable; those gates still need to pass. It means that the project should not claim mature production behavior across environments or provider configurations that have not been exercised. The first public tag should identify the reviewed source, and release notes should describe both implemented behavior and remaining limitations. A stable-version transition should be a separate evidence-based decision rather than a marketing consequence of publishing a repository.

Trusted publishing can reduce the need to store long-lived package-upload credentials in workflow secrets. The Python Packaging User Guide documents a GitHub Actions workflow using that mechanism [9]. TokenPropulsion prepares a protected publishing environment and limits the permissions of release jobs. Actual publisher registration is an external configuration step and must not be implied by the presence of a workflow file. GitHub release publication and PyPI publication are also distinct actions; completing one does not establish that the other has occurred.

Public assets need similar provenance. A repository hero image can improve discoverability without becoming evidence of algorithmic performance. Its metadata should state how it was generated, its actual dimensions, and whether an 8K deliverable was resized from a smaller native output. The whitepaper should be versioned alongside its editable source and reproducible PDF-generation procedure. Attractive presentation is compatible with scientific caution when illustrations are clearly illustrations and claims remain tied to inspectable reports.

Maintenance policy should preserve the project's narrow contract. Changes to protection rules, profiles, counting identity, or serialization can alter integration behavior even when the public function signature stays the same. Release notes should explain such changes with representative examples and relevant regression evidence. Contributors should avoid introducing implicit external calls or undocumented prompt retention. Any proposed learned compressor belongs behind a new explicit interface with its own dependency, privacy, quality, and performance evaluation requirements.

Finally, governance should make disagreement productive. Independent reviewers may reasonably prefer a broader parser, a smaller protection vocabulary, stricter refusal, or a different accounting strategy. The repository's value is that these choices can be examined and tested rather than hidden behind a generic optimization claim. Issues and pull requests should focus on reproducible behavior, threat scenarios, and measured tradeoffs. The goal is a tool that can be critically assessed and selectively adopted, including by teams that choose to use only its evaluation framework.

19. Research agenda and conclusion

The immediate research priority is application-specific validation. A small collection of reviewed prompt templates with strict correctness evaluators would provide stronger evidence than a much larger set of unlabelled strings. Such work should measure the proportion of inputs protected, the proportion actually changed, the distribution of real token differences, and the paired effect on task success. Results should include regressions and unchanged cases. A method that helps one template and harms another should be described at that resolution rather than collapsed into an unconditional recommendation.

A second priority is accounting extensibility without provenance loss. Local tokenizer plugins could improve offline measurement for explicitly supported vocabularies, provided that their model mapping, version, license, and exactness scope are recorded. Remote tokenization remains useful where official local support is absent, but its latency and privacy implications must stay visible. Pricing extensions should distinguish cached input, ordinary input, output, and other account-specific charges when verified data permits. More detailed arithmetic is useful only when the underlying usage and rate assumptions are trustworthy.

A third priority is comparative evaluation. Established learned compressors offer capabilities well beyond formatting normalization, and a fair study should acknowledge that difference rather than choose a favorable narrow benchmark. Comparisons should include quality, overhead, model dependencies, hardware, privacy requirements, and operational complexity. TokenPropulsion might serve as a conservative baseline or an admission layer within such a study. Whether that composition is useful is an empirical question. It should not be asserted from architectural compatibility alone.

The broader conclusion is that prompt optimization is an evidence-management problem as much as a text-transformation problem. Users need to know what changed, which invariants were checked, what the counter measured, what the experiment omitted, and how to recover the original input. The current implementation supplies a deliberately restricted contract around those questions. Its local tests, build artifacts, benchmark fixtures, and machine-readable reports make the contract inspectable. Its explicit unverified fields prevent missing remote evidence from being mistaken for favorable results.

TokenPropulsion therefore offers a foundation for cautious experimentation rather than a universal promise of shorter, equivalent prompts. The most responsible adoption path is incremental: review the admitted input class, reproduce the local checks, attach task-specific validation, verify provider behavior where relevant, and monitor the actual deployment objective. If that process finds no benefit, retaining the original prompt is a correct outcome. Measurable improvement is valuable only when the measurement and the behavior it describes remain worthy of trust.

20. Deployment decision protocol

An adopting team can turn the preceding analysis into a concrete acceptance record. The record should identify the intended workload, prompt-template owners, data classification, downstream model, tokenizer procedure, transformation profile, and success evaluator. It should also state which application actions remain outside the optimizer's authority. This prevents a favorable formatting experiment from silently expanding into permission for an agent to perform consequential operations. The acceptance record is a review instrument, not a certification supplied by this project.

The first decision is whether the workload admits any formatting change. Exact-output, regulated, security-policy, source-code, and structured-data workflows may appropriately require identity. For such workloads, TokenPropulsion can still provide accounting and an auditable refusal result, but an organization should not manufacture an optimization requirement merely to justify installing the tool. A documented

decision to bypass transformation can be as informative as a measured improvement, particularly when it preserves a constraint whose failure would be expensive.

The second decision concerns evidence thresholds. Before deployment, the team should choose acceptable task-success differences, failure categories that require immediate rejection, and a minimum practical benefit after overhead. Those thresholds should reflect the application rather than the benchmark's easiest cases. A small reduction may be irrelevant when requests are infrequent, while a formatting regression may be unacceptable in a high-consequence workflow even if average task success appears unchanged. Explicit thresholds reduce the temptation to reinterpret ambiguous results after observing them.

The third decision is operational containment. A gradual rollout can compare metadata without transmitting duplicate private prompts to an additional service. Where paired provider experiments are authorized, budget limits and reviewed datasets should precede execution. The team should know how to disable optimization, recover the original input, inspect sanitized failure records, and determine whether an upstream or downstream change invalidates the evaluation. Model revisions, tokenizer changes, template changes, and new document sources can all require renewed testing.

The fourth decision is ownership. Someone must maintain the protected-input policy, review dependency updates, handle vulnerability reports, and decide when evidence supports a new release. An unattended optimization component can accumulate risk even when its original implementation is small. The maintainer should preserve historical reports without confusing them with current guarantees, and should attach new evidence to the exact source and configuration evaluated. This practice makes later incident analysis possible without relying on memory or a marketing description.

Finally, adoption should remain reversible at the system level. The library preserves text, but application rollback also requires configuration control, deployment procedures, and clear responsibility for downstream effects. A team should be able to explain why a changed prompt was accepted and which evidence would cause that decision to be reconsidered. That explanation is the practical endpoint of auditable optimization: a measured, bounded choice whose assumptions remain visible throughout its operational life.

Independent replication should be welcomed even when it reports disappointing results. A public record of unchanged prompts, failed assumptions, and limited benefits can improve engineering decisions more than an unsupported success narrative. The repository is intended to make that scrutiny practical by publishing source, methods, artifacts, and limitations together, with claims that remain open to correction.

References

[1] Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. (2023). *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models*. EMNLP. https://aclanthology.org/2023.emnlp-main.825/ . DOI: 10.18653/v1/2023.emnlp-main.825.

[2] Pan, Z., et al. (2024). *LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression*. Findings of ACL. https://aclanthology.org/2024.findings-acl.57/ .

[3] Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics, 12, 157-173. https://aclanthology.org/2024.tacl-1.9/ . DOI: 10.1162/tacl_a_00638.

[4] Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., and Beutel, A. (2024). *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*. https://arxiv.org/abs/2404.13208 .

[5] Sennrich, R., Haddow, B., and Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL. https://aclanthology.org/P16-1162/ .

[6] MacFarlane, J., et al. *CommonMark Specification*, version 0.31.2. https://spec.commonmark.org/0.31.2/ . Accessed 17 October 2026.

[7] Bray, T., editor. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. https://www.rfc-editor.org/rfc/rfc8259 .

[8] xAI SDK maintainers. *Official xAI Python SDK*, including synchronous and asynchronous tokenization examples. https://github.com/xai-org/xai-sdk-python . Documentation: https://docs.x.ai/developers/model-capabilities/text/generate-text . Inspected SDK version 1.20.0; accessed 17 October 2026.

[9] Python Packaging Authority. *Publishing package distribution releases using GitHub Actions CI/CD workflows*. https://packaging.python.org/en/latest/guides/publishing-package-distribution-releases-using-github-actions-ci-cd-workflows/ . Accessed 17 October 2026.

Evidence artifacts and reproducibility

The primary implementation evidence is the repository source, tests, `docs/test-results.json`, `docs/coverage.json`, `docs/bandit.json`, `docs/dependency-audit.json`, `docs/install-smoke.json`, and `benchmarks/results/local.json`. These original project artifacts support implementation claims; external references provide context rather than substitute evidence for TokenPropulsion performance.

Run `python scripts/check_whitepaper.py` to verify the declared body length. To regenerate the PDF, install `docs/whitepaper/requirements-render.txt` and run `python scripts/render_whitepaper.py`. Fonts use system Arial or DejaVu Sans; exact pagination may vary with font version. The published PDF is the visually inspected output.